

How A Run Works

Here's the simple story.

Imagine one run

We start with a SWE-bench task that says:

- fix a bug in `src/user/email.js`
- run `test/user/emails.js`

Step 1: Task input

AgentBench loads the raw task data.

That includes things like:

- repo name
- bug description
- requirements
- tests to run
- workspace path

This is just the **problem statement** coming in.

Step 2: Formatted prompt

AgentBench turns that raw task into one clean message for the agent.

So instead of scattered dataset fields, it becomes something like:

- here is the repo
- here is the bug
- here are the requirements
- here are the tests
- here is your writable workspace
- inspect the repo, make changes, validate them

This is the first real **prompt**.

Step 3: System context

Deep Agents also has its own higher-level instructions.

That tells the model things like:

- you are a coding agent
- you can use tools
- inspect files before changing them
- don't guess if you can verify

So now the model has:

- a **system prompt** saying how to behave
- a **user prompt** describing the specific bug

Step 4: Final model request

The wrapper sends a real chat-completions request to Dynamo.

That request includes:

- model name
- system prompt
- user prompt
- tool definitions
- tool choice mode, like `auto`

So at this point the model is no longer just seeing plain text. It is seeing:

- the problem
- the instructions
- the list of tools it can call

Step 5: Tool exposure

Suppose the tool list includes:

- `ls`
- `read\_file`
- `edit\_file`
- `execute`

Now the model can decide:

- "I should call `ls` first"
- "then `read\_file`"
- "then maybe `edit\_file`"
- "then `execute` to run tests"

This is what I mean by:

- **what tools the model actually saw**

Step 6: Frontend/runtime preprocessing

Before the worker generates tokens, Dynamo preprocesses the request.

It may:

- tokenize the prompt
- inject tool formatting into the model's expected template
- enable the tool parser like `hermes`
- prepare the request for SGLang

This is the "translation layer" between:

- OpenAI-style request
- and
- model-ready request

Step 7: Model response

Now the model responds.

A healthy path might look like:

1. model returns a real tool call:
- `ls(path=...)`
2. tool runs
3. model sees the result
4. model returns another tool call:
- `read\_file(path=...)`
5. tool runs
6. model eventually calls:
- `edit\_file(...)`
7. tool runs
8. model calls:
- `execute(command="npm test ...")`
9. tool runs
10. model gives a final summary

So the run becomes:

- think
- inspect
- edit
- test
- summarize

Step 8: Final artifacts

At the end, AgentBench saves:

- the final response text
- the tool transcript
- measurements
- runtime logs
- git diff / patch
- lifecycle trace
- prompt lineage

So later you can answer:

- what problem did we give it?
- what exact prompt did we build?
- what tools were attached?
- did the model actually call tools?
- what happened at runtime?
- did it really edit code or just talk?

How to Read `json\_keys\_before` and `json\_keys\_after`

In `prompt\_evolution\_report`, these two columns show how the data shape changes from stage to stage.

- `json\_keys\_before`
- shows the object shape before the current stage
- `json\_keys\_after`
- shows the object shape after the current stage

They do not show the full values. They only show the structure using keys, arrays, and nested objects.

For example, you might see something like:

```
{
  "messages": [
    { "role": "...", "content": "..." }
  ],
  "request_context": { "request_id": "...", "phase": "...",
    "agent_hints": { "priority": "...",
      "tool_choice": "..."
    }
  }
}
```

That means:

- the structure now contains a `messages` array
- each message has keys like `role` and `content`
- the request also carries nested objects like `request\_context` and `agent\_hints`

So these columns let you see:

- what the shape looked like before a stage
- what the shape looked like after a stage

- whether the prompt became a request object
- whether tool/runtime fields were added
- when the request shape finally became a response/outcome shape

In simple terms:

- `json\_keys\_before/after` show the **object-shape evolution** of the run
- they back up the stage summaries with a more concrete structure view

Very short version

A typical prompt lineage is:

1. raw bug report comes in
2. AgentBench turns it into a clean task prompt
3. Deep Agents adds its coding-agent instructions
4. Dynamo receives the final request with tools
5. frontend preprocesses it for the model
6. model calls tools, reads files, edits code, runs commands
7. final response and patch are saved

That is the whole pipeline in simple terms.